

Ray Tracing

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

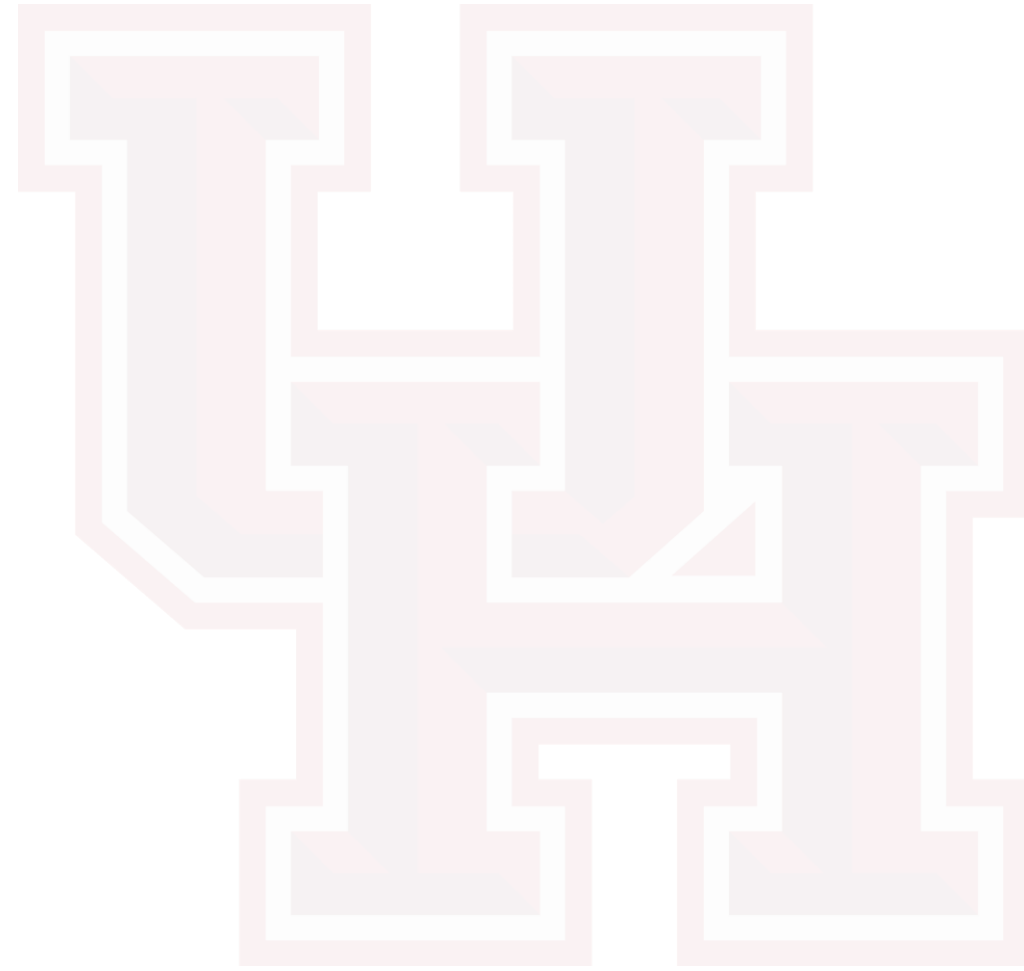
Cullen College of Engineering

University of Houston

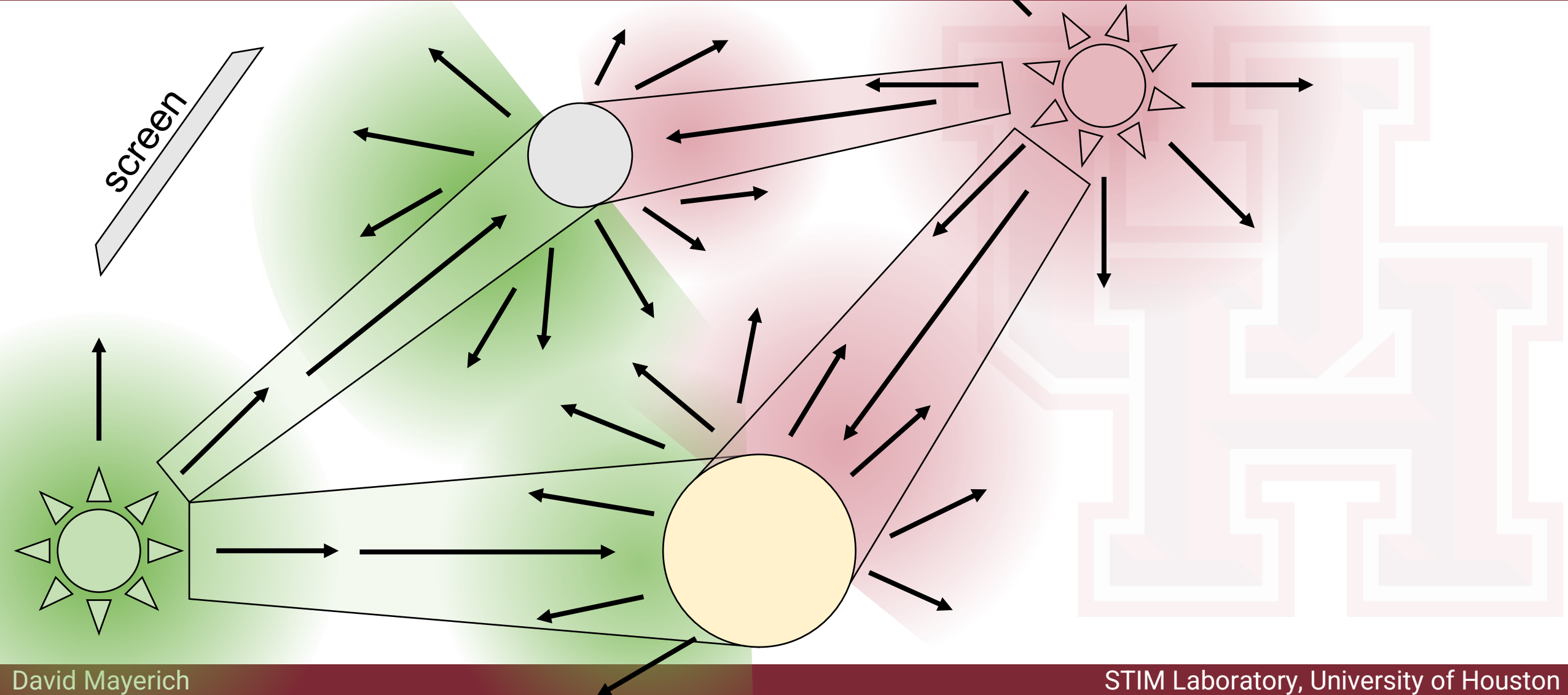
UNIVERSITY of **HOUSTON** | ECE

Ray Tracing: Goals

- Category of algorithms used to synthesize digital images
- Determine pixel color by back-tracking the light path
- Produces realistic images of complex materials
- Computationally expensive – computing resources increase dramatically with image complexity
- Potentially very parallel: each pixel in the image is independent

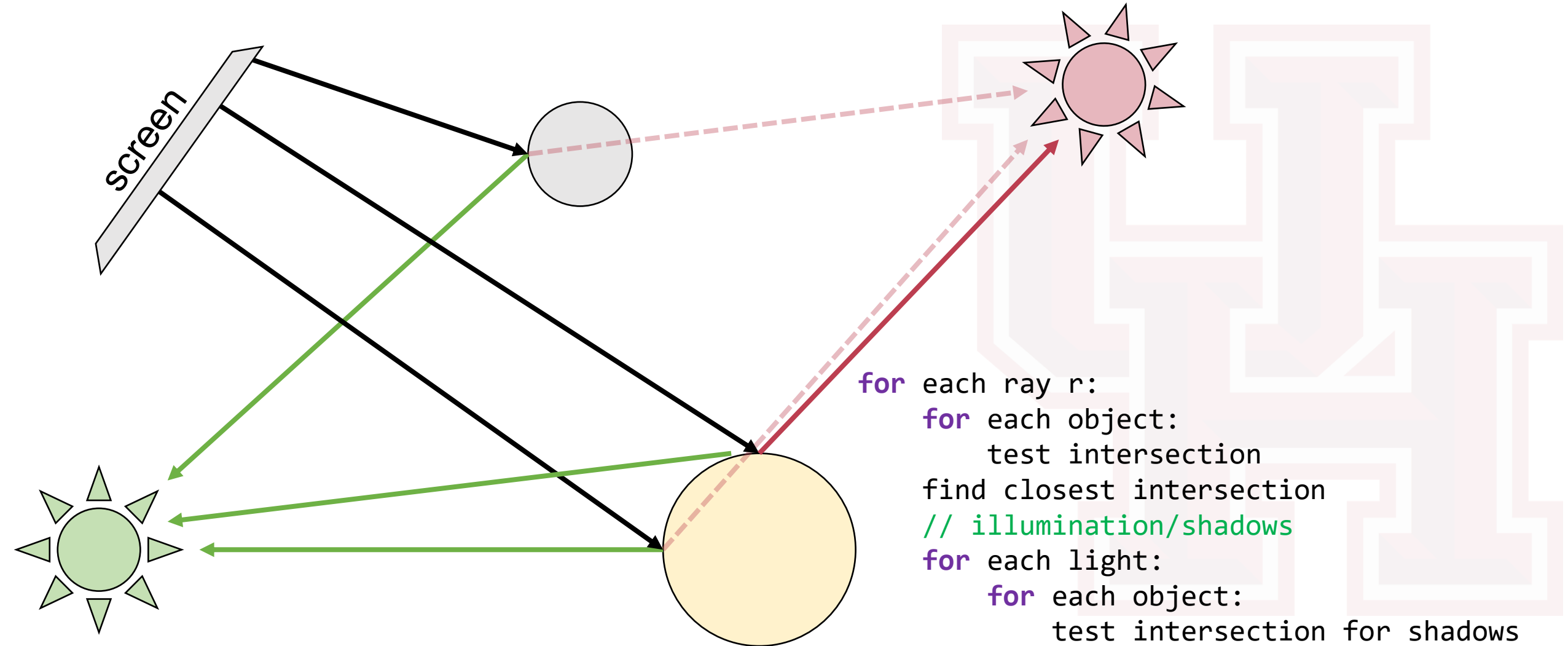


Light Scattering: “Forward Model”



The diagram illustrates a Mach-Zehnder interferometer. A green light source at the bottom left emits a beam that is split by a beam splitter (a small circle) into two paths. One path is reflected upwards by a mirror, and the other path is reflected downwards by another mirror. The two paths recombine at a second beam splitter (a larger yellow circle). The resulting single beam is directed towards a screen at the top left, labeled "screen". A bracket indicates the region "light arriving at a detector" on the screen. A red sun-like symbol is located at the top right, and a large, faint, stylized letter 'I' is in the background.

Ray Tracing: “Inverse Model”

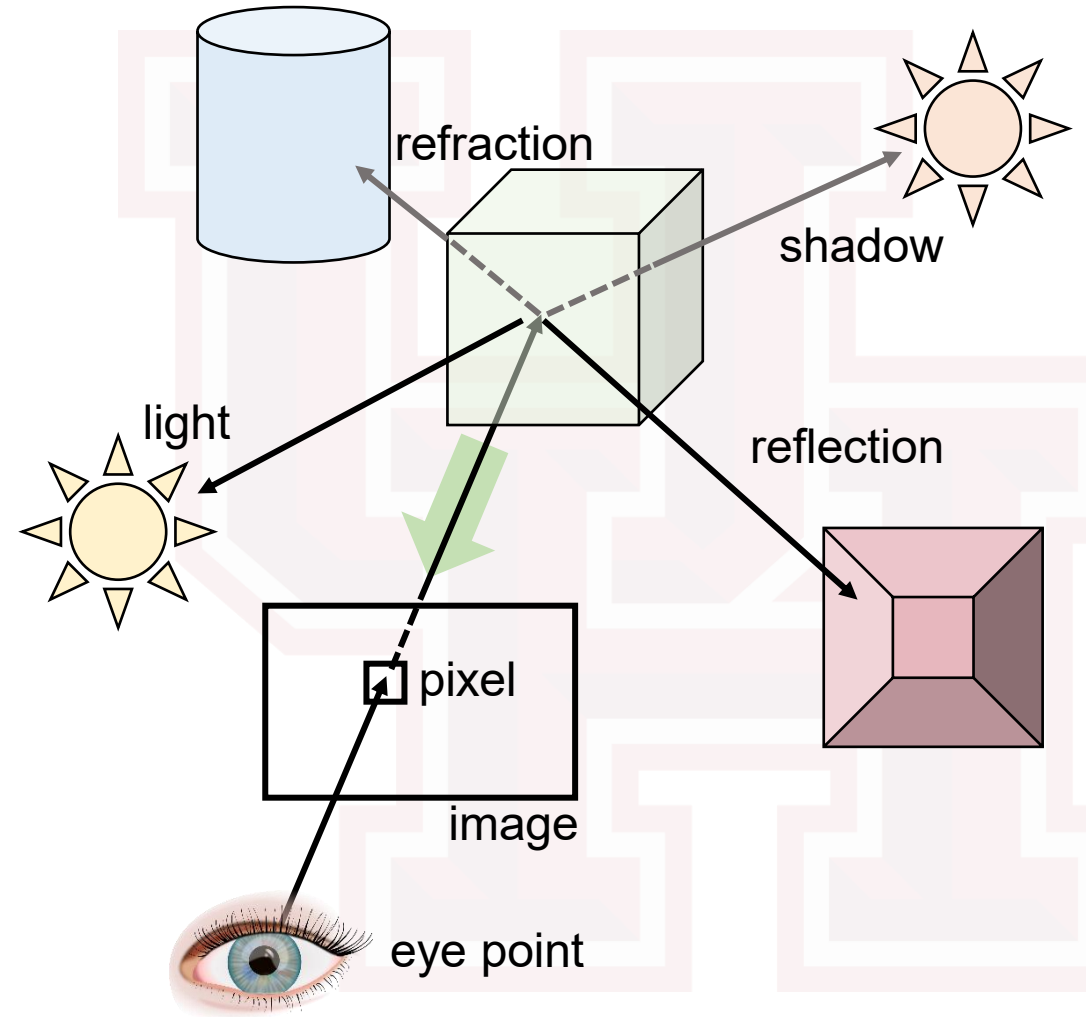


Ray Tracing Algorithm

for each pixel:
 find all ray intersections
 calculate closest intersection
 compute color at the intersection

$$C_i = C_e + C_d + k_r C_r + k_t C_t$$

- C_i is the light at the intersected object
- C_e is the emission from the object
- C_d is the direct illumination from incident light
- C_r is the light reflected from the object
- C_t is the light transmitted from the object
- k_r and k_t are material properties for reflection and transmission



Recursive Ray Tracing

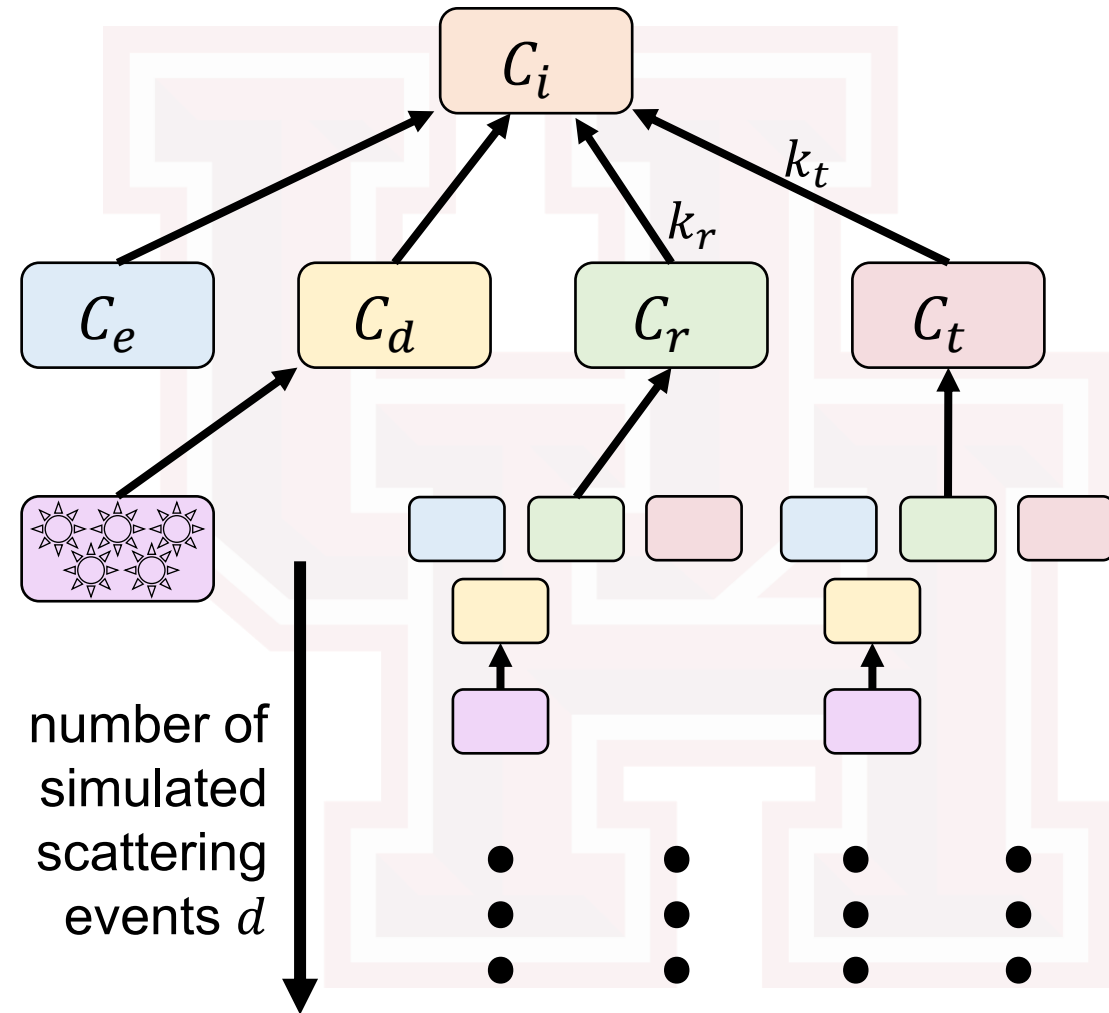
$$C_i = C_e + C_d + k_r C_r + k_t C_t$$

- Assume N objects, L lights, and P rays
 - Detecting the nearest object is $O(N)$
 - C_d tests intersections between lights and objects: $O(NL)$
 - C_r and C_t require recursion:

$$O(P[N + NL + R + T])$$

$$\text{where } R = T = N + NL + R' + T'$$

$$O(2^{d+1}[N + NL])$$



Recursive Ray Tracing

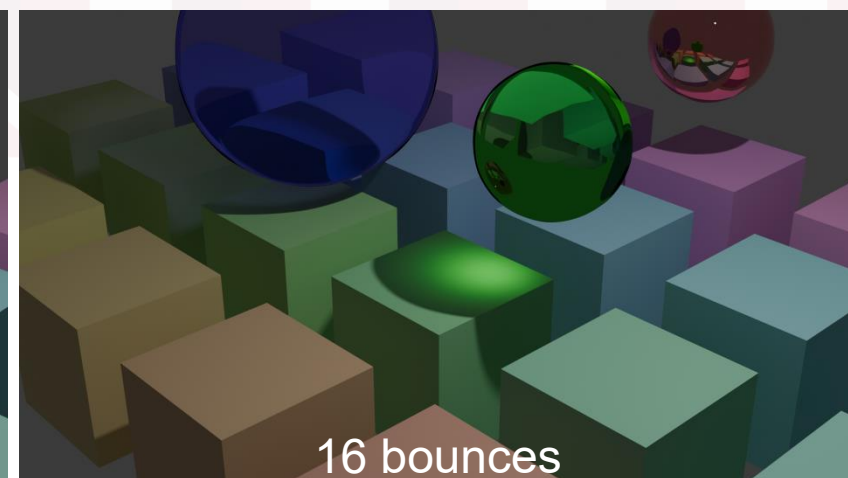
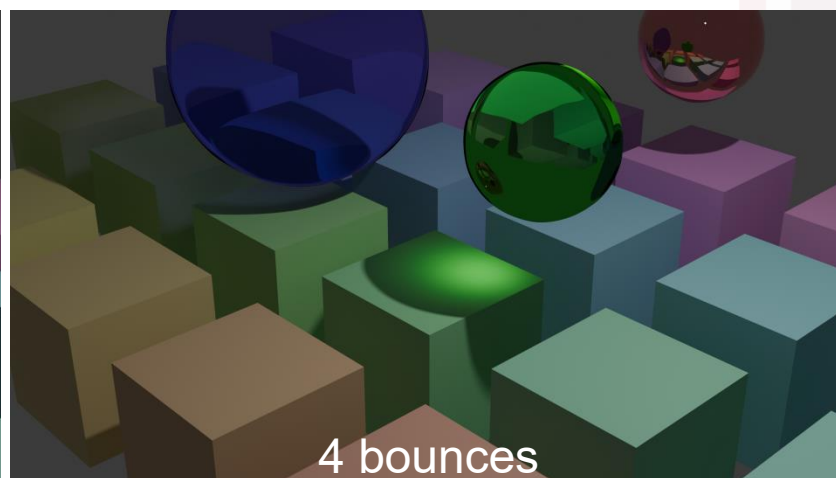
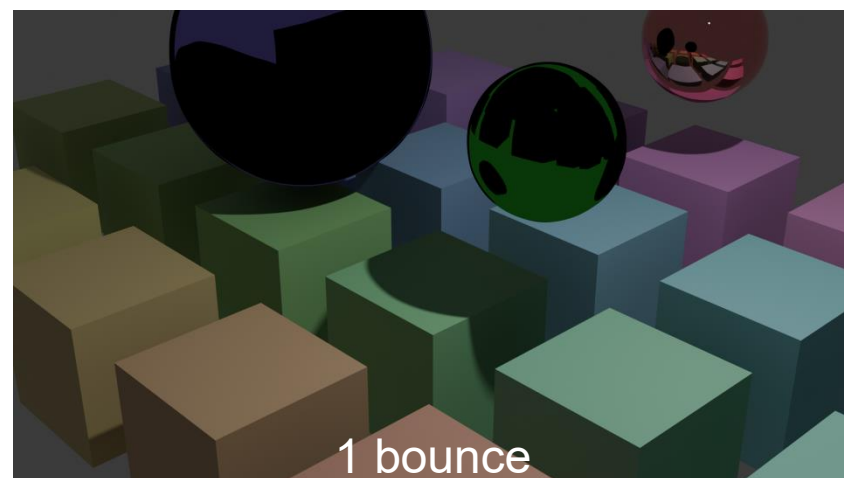
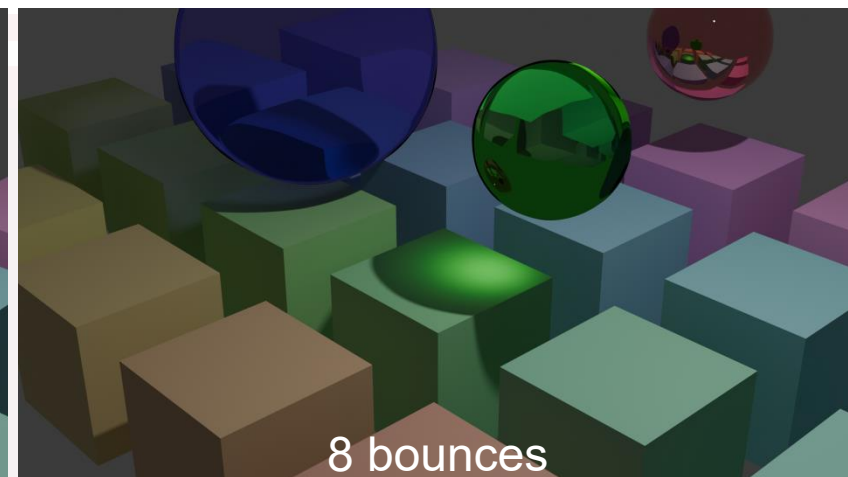
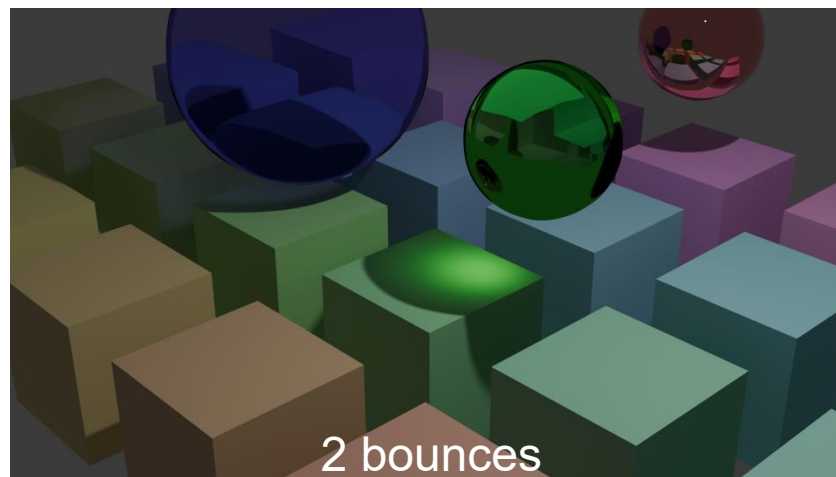
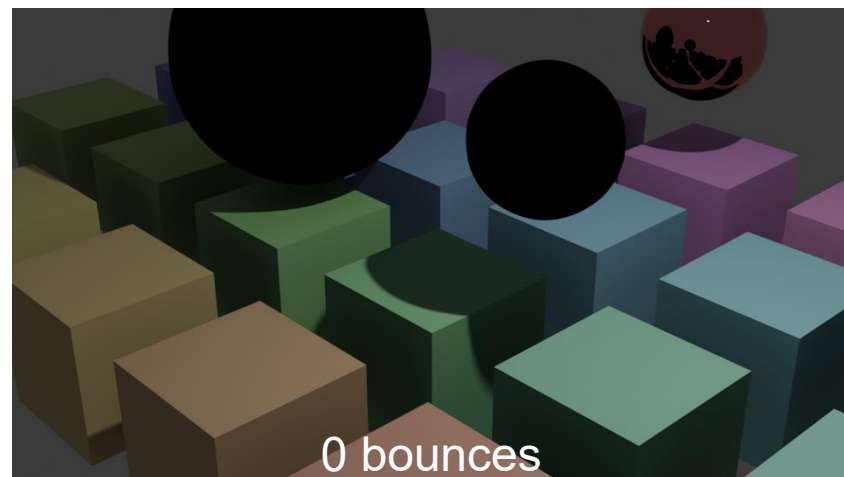
```
for each pixel ray r:  
    color pixel = LaunchRay(r, 0)  
  
LaunchRay(r, b++):  
    if(b > BOUNCE_MAX) return black;  
    for each object: test intersection  
    for each light: l_color += ShadowRay(light)  
    r_color = LaunchRay(reflection, b++)  
    t_color = LaunchRay(transmission, b++)  
    return l_color + r_color + t_color
```

```
ShadowRay(r):  
    for each object:  
        if intersection, return false  
    return true
```



Obligatory ray-tracing example by Gilles Tran, oyonale.com
POVray (povray.org)

Recursive Ray Tracing



*rendered using Blender with Cycles ray tracing

Recursive Ray Tracing

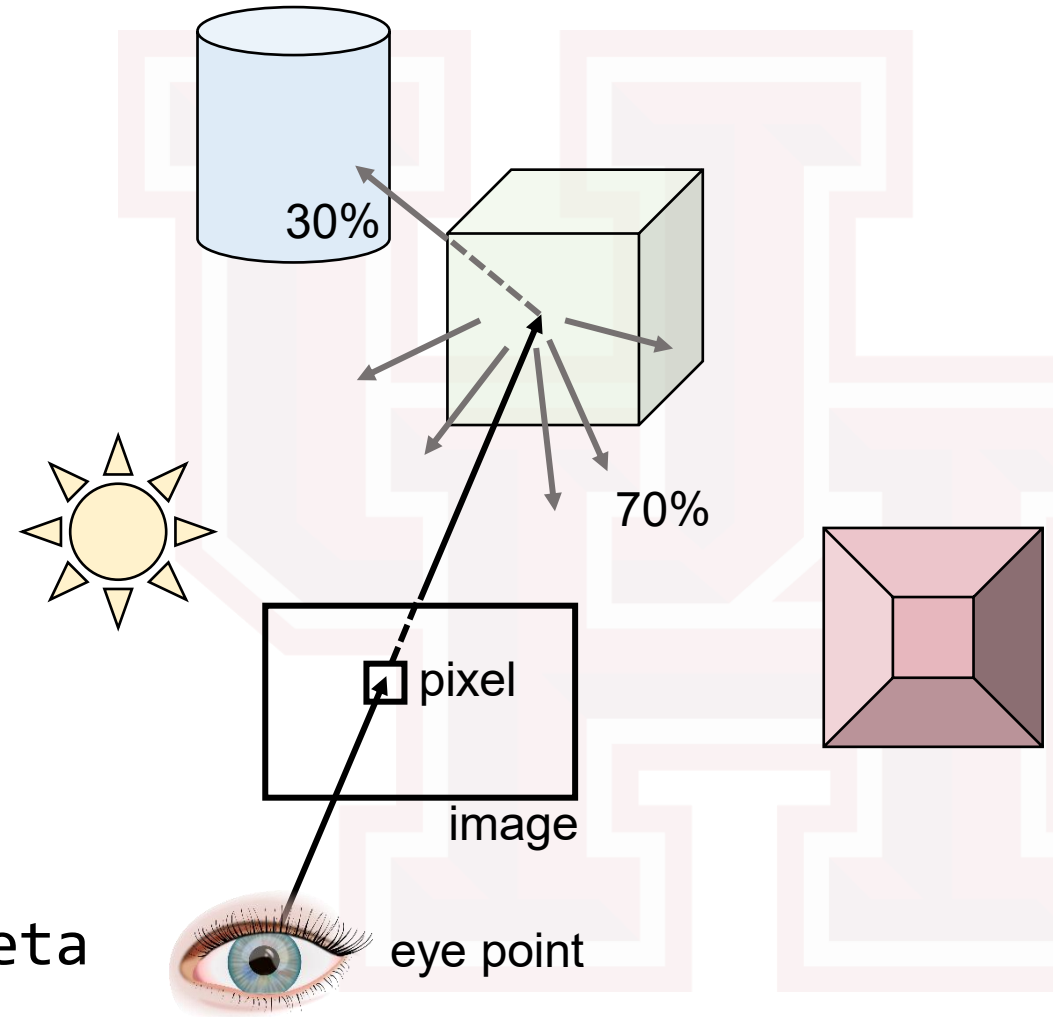
*rendered using Blender with Cycles ray tracing



Path Tracing

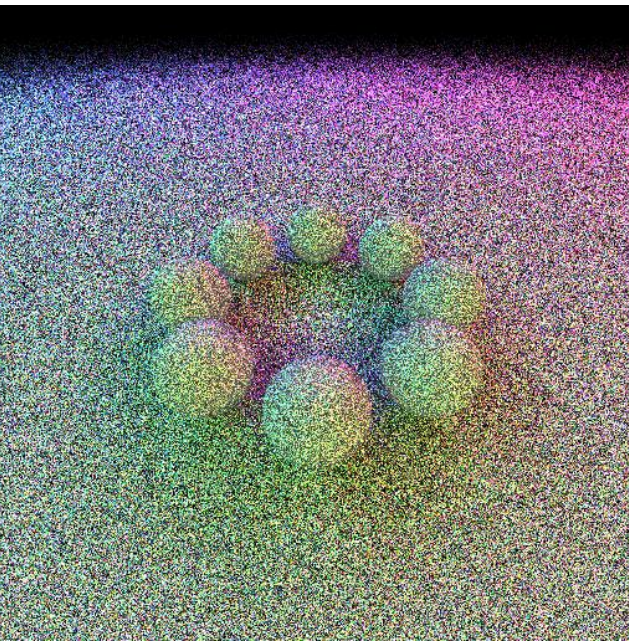
- Statistically sample individual ray paths
- ```
for each pixel ray r:
 color pixel = LaunchRay(r, 0)
```

```
LaunchRay(r, b):
 if(b > BOUNCE_MAX) return black;
 entity = find_nearest_entity(r)
 color_emit = entity.emission()
 next_ray = entity.scatter()
 cos_angle = cosine(r, next_ray)
 color_in = LaunchRay(next_ray, b++)
 return color_emit + color_in * cos_theta
```

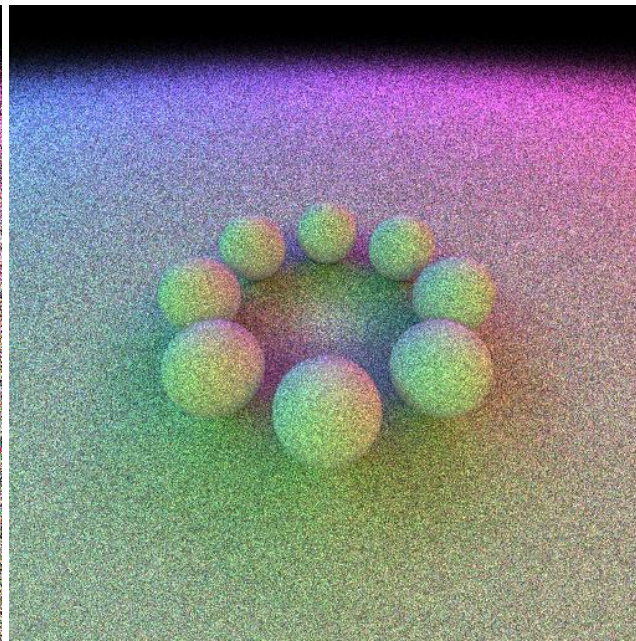


# Path Tracing

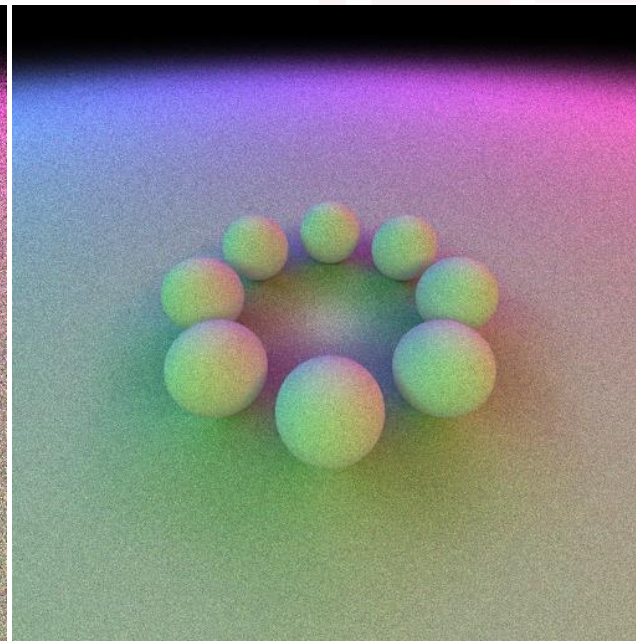
- Average several paths instead of one “deep” recursive tree
- Trade evaluation time for noise
- Solve using Monte-Carlo integration



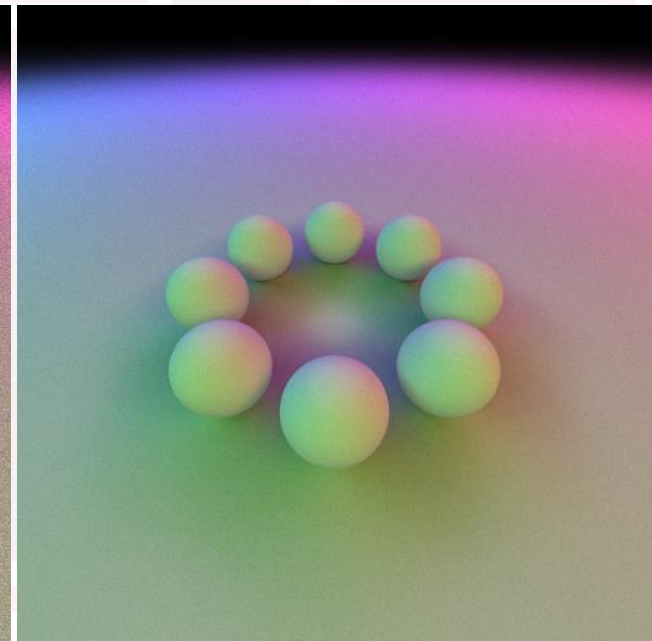
100 samples



1,000 samples



10,000 samples



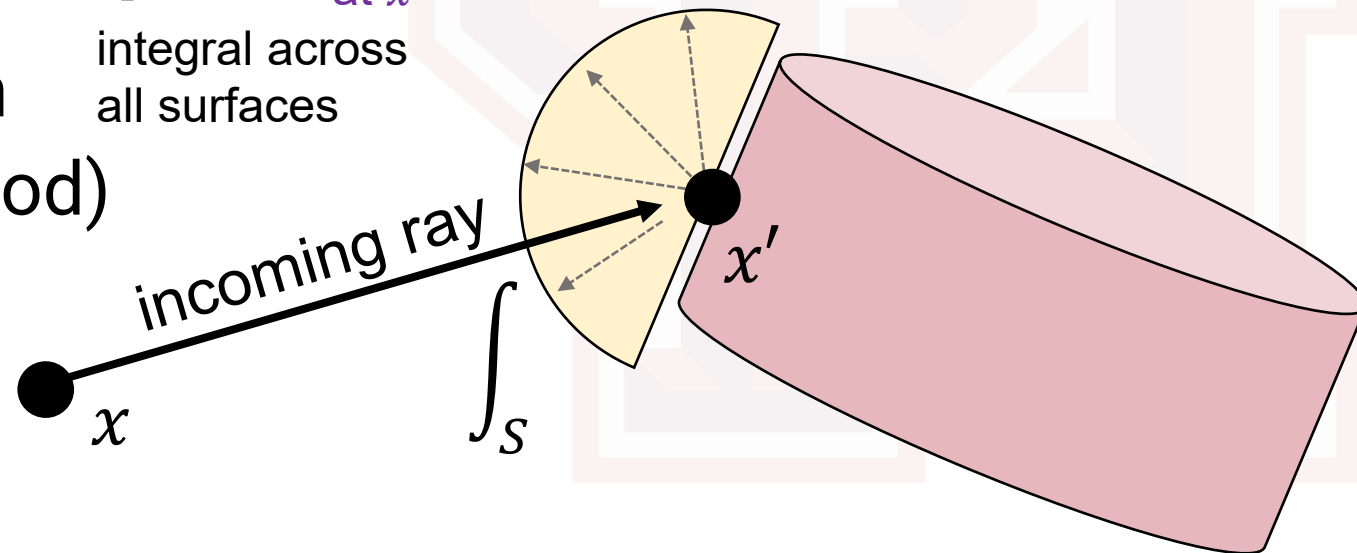
100,000 samples

# Rendering Equation

$$\underbrace{I(x, x')}_{\text{light intensity from } x' \text{ to } x} = \underbrace{g(x, x')}_{\text{geometry term}} \left[ \underbrace{\epsilon(x, x')}_{\text{emitted light from } x' \text{ to } x} + \int_S \underbrace{p(x, x', x'')}_{\text{intensity of light scattered from } x'' \text{ to } x \text{ by the patch at } x'} \underbrace{I(x', x'')}_{\text{light intensity from } x' \text{ to } x} dx'' \right]$$

↑  
integral across all surfaces

- Approximates Maxwell's Equation
- Geometric optics (Luneburg Method)
- Solve the integral by:
  - Recursing through  $I(a, b)$
  - Use Monte-Carlo integration

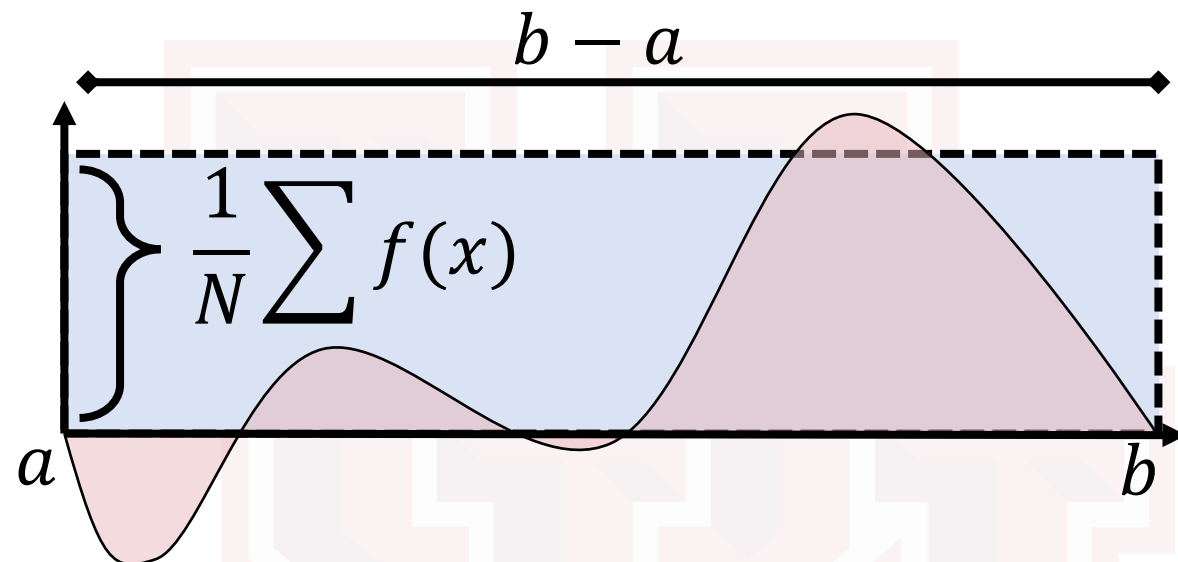


# Monte-Carlo Integration

- Numerically solve an integral using random numbers:

$$\int_a^b f(x) = \frac{b-a}{N} \sum_{i=0}^{N-1} f(x_i)$$

- The integral of a function can be calculated from its mean
- Approximate the mean using random sampling



- Advantage: The error of the approximation decreases with:

$$\epsilon \approx \frac{1}{\sqrt{N}}$$

(independent of the dimension)

# Vector Math – points and vectors

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \quad \mathbf{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

- subtracting 2 points produces a vector:

$$\mathbf{v} = \mathbf{p}_1 - \mathbf{p}_0$$

- dot products:

$$c = \mathbf{v}_0 \cdot \mathbf{v}_1$$

- cross products:

$$\mathbf{v}_2 = \mathbf{v}_0 \times \mathbf{v}_1$$

- adding a point and vector produces a new point:

$$\mathbf{p}_1 = \mathbf{p}_0 + \mathbf{v}_0$$

- Ray

$$\mathbf{r}(t) = \mathbf{p} + \mathbf{v}t$$

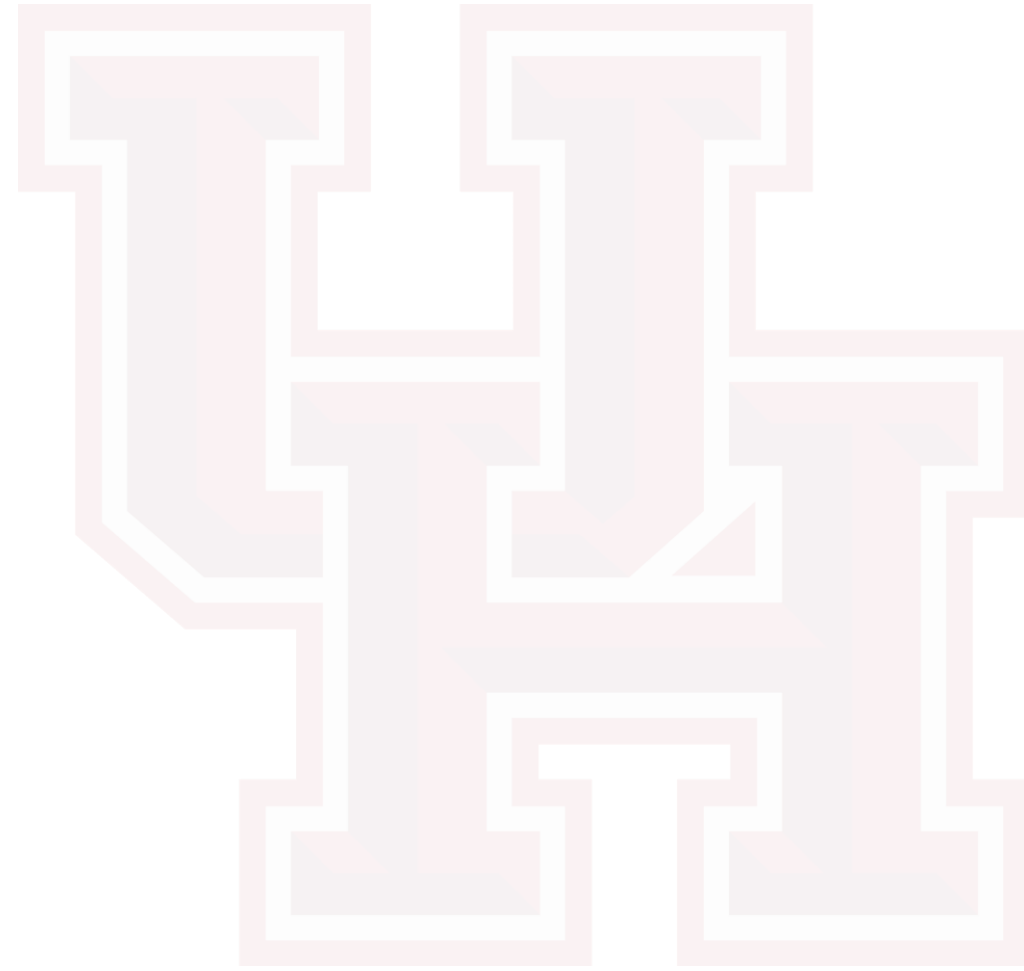
- produces a new point by parameterizing an origin  $\mathbf{p}$  and direction vector  $\mathbf{v}$

# Other Structures

- Colors

$$\mathbf{c} = [r \quad g \quad b]$$

- identical representation to vectors
- different operations



# Other Structures

## Colors

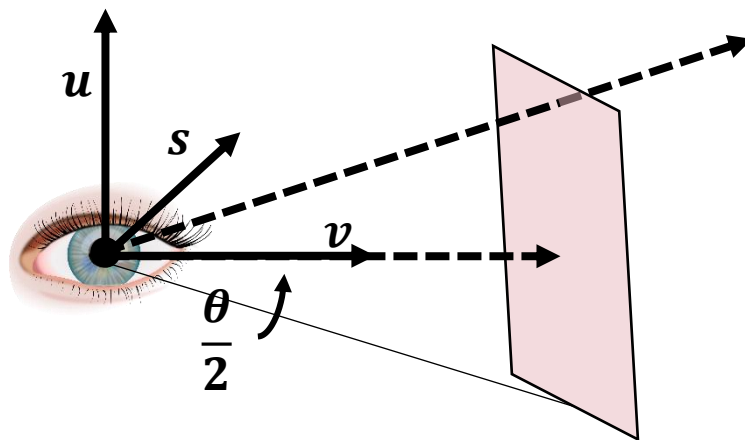
$$c = [r \quad g \quad b]$$

- element-wise multiplication:

$$c_2 = c_0 * c_1$$

- addition

$$c_2 = c_0 + c_1$$



## Camera

- eye point  $p$
- view direction  $v$
- "up" vector  $u$
- "side" vector  $s = u \times v$
- field of view  $\theta \approx 30^\circ$
- resolution  $R$  (ex.  $500 \times 500$  pixels)

$$\tan \frac{\theta}{2} = \frac{1}{d} \quad \Delta_x = \Delta_y = \frac{1}{R}$$
$$r = d v + \left( p_x - \frac{R}{2} \right) s + \left( p_y - \frac{R}{2} \right) u$$

where  $p = (p_x, p_y)$  is a pixel position

# Object Intersections

ray:  $\mathbf{r}(t) = \mathbf{p} + \mathbf{v}t$

sphere:  $(\mathbf{s} - \mathbf{a}) \cdot (\mathbf{s} - \mathbf{a}) = r^2$

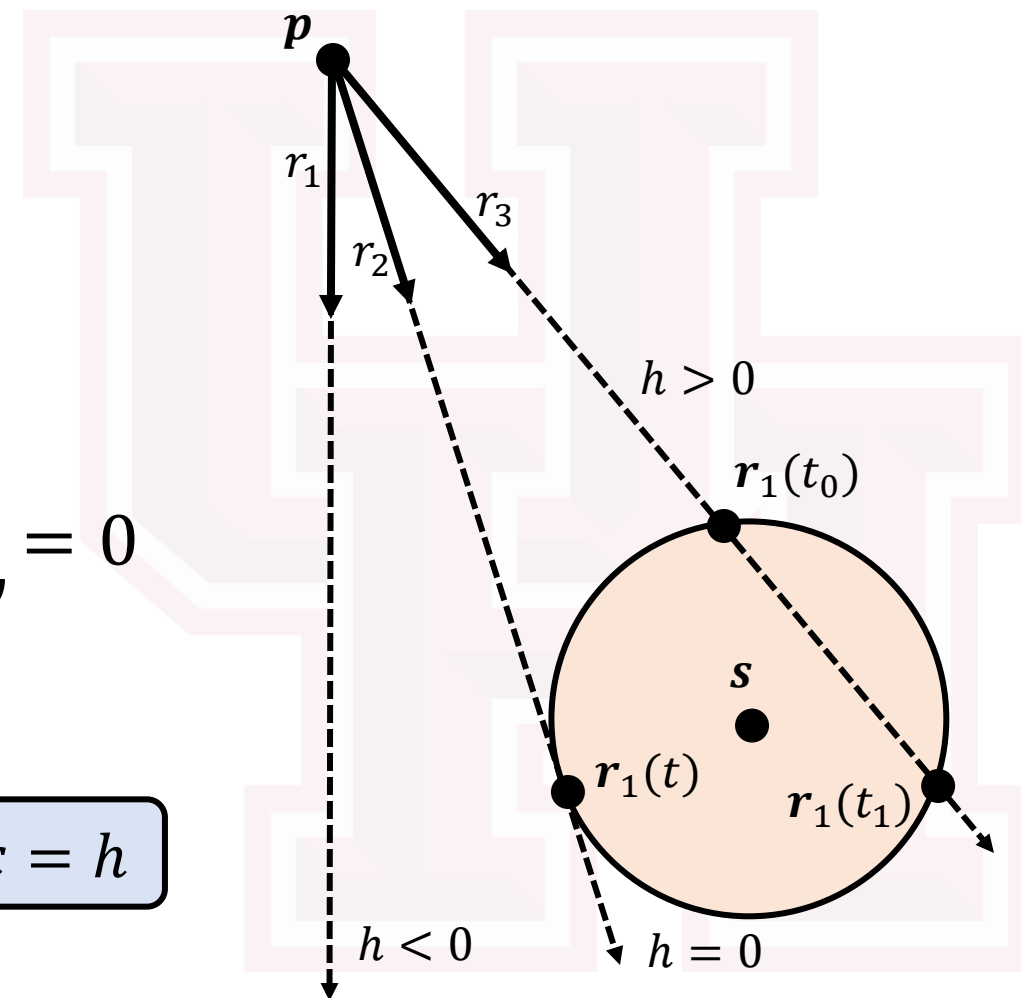
- Find a point on  $\mathbf{r}(t)$  such that:  
$$[\mathbf{s} - (\mathbf{p} + \mathbf{v}t)] \cdot [\mathbf{s} - (\mathbf{p} + \mathbf{v}t)] = r^2$$

- Expands to a quadratic equation:

$$\underbrace{\mathbf{v} \cdot \mathbf{v}}_a t^2 + \underbrace{2\mathbf{v} \cdot (\mathbf{s} - \mathbf{p})}_b t + \underbrace{(\mathbf{s} - \mathbf{p}) \cdot (\mathbf{s} - \mathbf{p}) - r^2}_c = 0$$

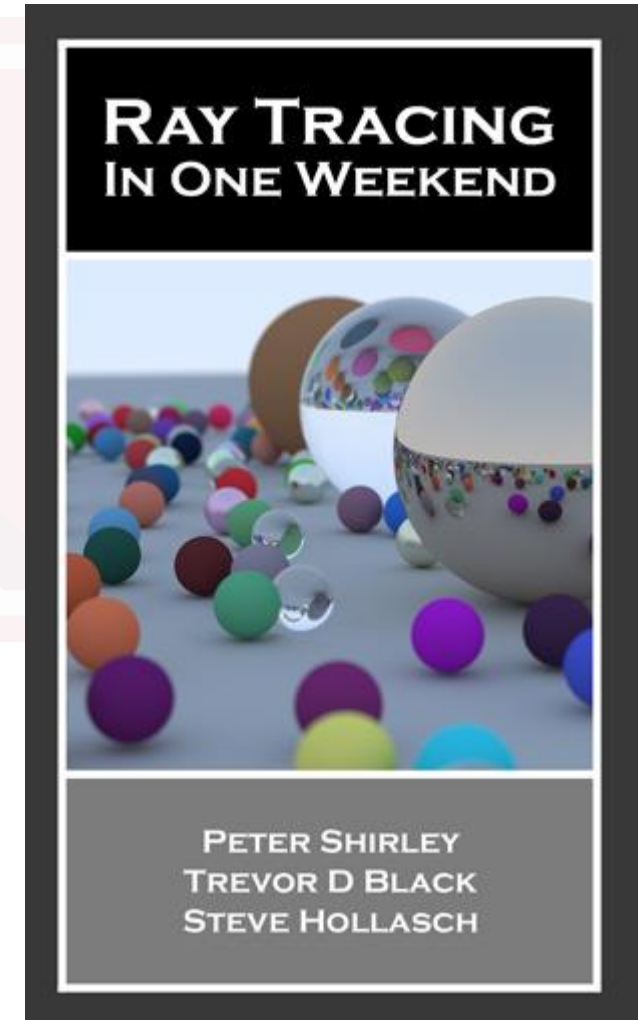
- Solve using the quadratic equation:

$$t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \rightarrow b^2 - 4ac = h$$



# Vector Math Implementation

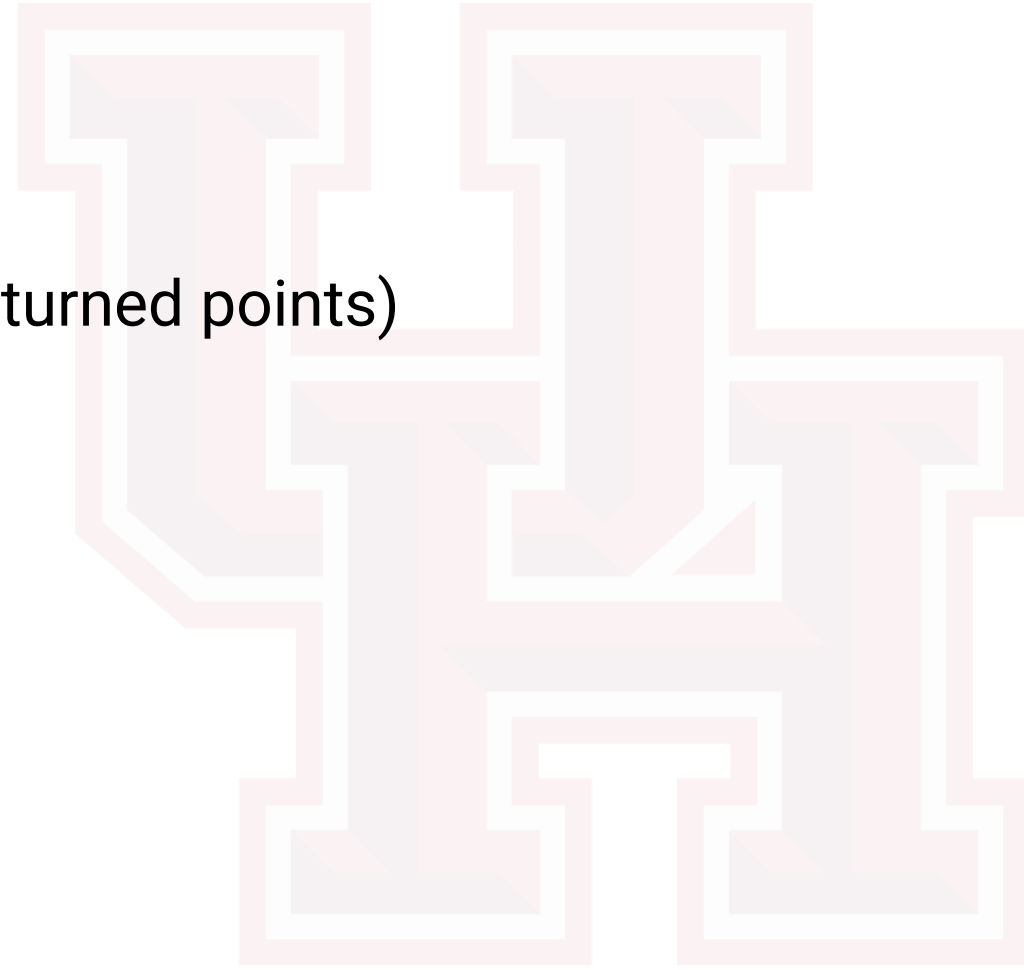
1. OpenGL Mathematics (GLM):
  - <https://github.com/g-truc/glm>
  - fast and has all required operations
  - documentation isn't great
  - aliases make debugging harder
2. Ray Tracing in One Weekend as a starting point:
  - <https://raytracing.github.io>
  - describes exactly what is needed for ray tracing
  - good explanations
  - can copy/paste as needed



# Vector Math Implementation

## 3. My implementation:

- separate classes for vectors, points, and colors
- overloaded operators used for each type
- returned correct types (ex. vector differences returned points)
- started from Ray Tracing in One Weekend



# Programming Assignment 2

- Implement ray tracing of one sphere
- Color the sphere based on its normal:

$$c = |v|$$

$$r = |x| \quad g = |y| \quad b = |z|$$



- This is all covered in Ray Tracing in One Weekend (Chapters 2-6)
- Use Programming Assignment 1 as a starting point
  - Writing to `output_image_ptr` will immediately show up on the screen
  - You can start with a fixed camera but moving it can help you debug
- This is a “screenshot assignment”